

Struct

- C dilinde bazen bir varlığa ait birden fazla bilgiyi birlikte tutmamız gerekir.
- Örneğin bir öğrencinin:
 - adı
 - numarası
 - notu
 - yaşı

Struct

- Bu bilgileri ayrı ayrı değişkenlerde tutmak mümkündür; ancak bu yöntem:
 - karmaşıklık oluşturur,
 - kod okunabilirliğini düşürür,
 - veri yönetimini zorlaştırır.
- Bu nedenle struct kullanılır.
- struct, farklı veri tiplerini tek bir çatı altında toplar.
- Yani struct, “bir kayıt yapısı” gibi düşünülebilir.

Struct

- Aynı varlığa ait verileri gruplayarak daha düzenli programlar yazılır.
- Özellikle kayıt işlemleri, veri tabanı mantığı, öğrenci/personel bilgileri gibi yerlerde çok kullanılır.
- Struct, C dilinde kullanıcı tanımlı veri yapılarının en temelidir. Diziler aynı türden verileri saklarken, struct farklı türleri aynı yapıda bir araya getirir.

Struct

Bir struct genel olarak şu şekilde tanımlanır:

```
struct Ogrenci {  
    char ad[20];  
    int no;  
    float not;  
};
```

Bu tanımdan sonra struct türünde değişken tanımlanabilir:

```
struct Ogrenci ogr1;
```

Alanlara erişim için . operatörü kullanılır:

```
ogr1.no = 101;  
ogr1.not = 85.5;
```

Struct

```
#include <stdio.h>
```

```
struct Ogrenci {  
    char ad[20];  
    int no;  
    float not;  
};
```

```
int main() {  
    struct Ogrenci ogr1;  
  
    ogr1.no = 101;  
    ogr1.not = 90.5;  
    strcpy(ogr1.ad, "Ali");  
    printf("Numara: %d\n", ogr1.no);  
    printf("Not: %.2f\n", ogr1.not);  
    printf("Adı: %s\n", ogr1.ad);  
  
    return 0;  
}
```

- struct Ogrenci bir tür tanıımıdır.
- ogr1 bu türden bir değişkendir.
- Alanlara erişimde . kullanılır.

Struct

Bir öğrenciyi temsil eden struct Ogrenci yapısını tanımlayınız. Bu yapı içinde aşağıdaki alanlar bulunsun:

- no
- ad

Programda:

1. Bir adet struct Ogrenci değişkeni oluşturunuz.
2. Öğrencinin numarasını ve adını **kullanıcıdan alınız.**
3. Girilen öğrencinin numarasını ve adını ekrana yazdırınız.

```
#include <stdio.h>
struct Ogrenci {
    int no;
    char ad[20];
};
int main(void) {
    struct Ogrenci ogr1;
    printf("Ogrenci numarasini giriniz: ");
    scanf("%d", &ogr1.no);
    printf("Ogrenci adini giriniz: ");
    scanf("%19s", ogr1.ad);
    printf("\nNo: %d\n", ogr1.no);
    printf("Ad: %s\n", ogr1.ad);
    return 0;
}
```

Typedef

- typedef, var olan bir veri tipine yeni bir isim vermek için kullanılır.
- Bu sayede daha okunabilir ve daha kısa kod yazılır.
- Özellikle uzun yazımlarda büyük kolaylık sağlar.
- Kod sadeliği sağlar.
- Tekrarları azaltır.
- Kullanıcı tanımlı veri tiplerini daha kolay kullanılabilir hale getirir.

Typedef

- Genel kullanım:
`typedef eski_tip yeni_isim;`

Basit örnek:

typedef int TamSayi;

Kullanım:

TamSayi x = 5;

Typedef

Struct ile kullanım:

```
typedef struct {  
    char ad[20];  
    int no;  
    float not;  
} Ogrenci;
```

Artık şöyle kullanılabilir:

```
Ogrenci ogr1;
```

```
struct Ogrenci {  
    char ad[20];  
    int no;  
    float not;  
};  
  
// Kullanımı:  
struct Ogrenci ogr1;
```

Union

- union, struct'a benzer bir kullanıcı tanımlı veri yapısıdır.
- Farklı türde verileri tek yapı içinde tanımlamaya izin verir.
- Ancak struct'tan önemli bir farkı vardır:

Struct:

- Her alan bellekte ayrı yer kaplar.

Union:

- Tüm alanlar bellekte aynı bölgeyi paylaşır.
- Yani union içinde aynı anda tüm alanlar ayrı ayrı tutulmaz.
- En son hangi alana değer verildiyse o anlamlıdır.

Union

Union tanımı:

```
union Veri {  
    int i;  
    float f;  
    char c;  
};
```

Kullanım:

```
union Veri v;  
v.i = 10;
```

Union

```
#include <stdio.h>
union Veri {
    int i;
    float f;
    char c;
};
int main() {
    union Veri v;
    v.i = 65;
    printf("i = %d\n", v.i);
    v.c = 'A';
    printf("c = %c\n", v.c);
    return 0;
}
```

Union

Özellik	Struct	Union
Bellek kullanımı	<u>Alanlar ayrı yer kaplar</u>	<u>Alanlar aynı belleği paylaşır</u>
Aynı anda kullanım	<u>Tüm alanlar kullanılabilir</u>	<u>Genelde tek alan güvenli</u>
Amaç	Veri gruplama	Bellek paylaşımı

Enum

- enum, sabit tamsayılara anlamlı isimler vermek için kullanılır.
- Kod içinde 0,1,2,3 gibi sayılar yerine açıklayıcı isimler kullanılır.
- Bu da kodun okunabilirliğini artırır.

Enum

```
#include <stdio.h>
```

```
enum Durum {PASIF, AKTIF, BEKLEMEDE};
```

```
int main() {  
    enum Durum d;  
    d = AKTIF;
```

```
    printf("Durum degeri: %d\n", d);
```

```
    return 0;  
}
```

```
#include <stdio.h>  
  
// Her bir durumu manuel olarak sayılar  
#define PASIF 0  
#define AKTIF 1  
#define BEKLEMEDE 2  
  
int main() {  
    int d; // Artık 'Durum' diye bir ti  
    d = AKTIF;  
  
    printf("Durum degeri: %d\n", d);  
  
    return 0;  
}
```



```
#include <stdio.h>
#include <string.h>
typedef enum {AKTIF, PASIF} Durum;
typedef struct {
    int id;
    char ad[20];
    Durum durum;
} Kullanici;
int main(void){
    Kullanici k1;
    k1.id = 1;
    strcpy(k1.ad, "Mehmet");
    k1.durum = AKTIF;
    printf("ID: %d\n", k1.id);
    printf("Ad: %s\n", k1.ad);
    printf("Durum: %d\n", k1.durum);
    return 0;
}
```

Struct Dizileri

- Tek bir struct değişkeni bir nesneyi temsil eder.
- Birden fazla öğrenci, kitap, çalışan gibi kayıt tutmak için struct dizileri kullanılır.
- Böylece aynı yapıya sahip çok sayıda veri düzenli biçimde saklanır.

```
struct Ogrenci {  
    int no;  
    char ad[20];  
    float ortalama;  
};
```

struct Ogrenci sinif[3]; //3 tane Ogrenci yapısı kadar yer ayır.

Struct Dizileri

Struct Dizilerinde Alanlara Erişim

- Dizi elemanına önce indeks ile ulaşılır.
- Sonra . operatörü ile alan erişilir.
- Bu yapı, tablo mantığında veri tutmayı kolaylaştırır.

```
sinif[0].no = 101;  
strcpy(sinif[0].ad, "Ali");  
sinif[0].ortalama = 85.5;
```

- `sinif[0]` → ilk öğrenci
- `sinif[0].ad` → ilk öğrencinin adı

Struct Dizileri

Struct Dizileri ile Döngü Kullanımı

- Struct dizileri genellikle for döngüsü ile gezilir.
- Bu sayede toplu veri girişi, listeleme, arama ve güncelleme yapılabilir.

```
for (int i = 0; i < 3; i++) {  
    printf("%d %s %.2f\n",  
        sinif[i].no,  
        sinif[i].ad,  
        sinif[i].ortalama);  
}
```

Struct Dizileri

Bir sınıftaki 2 öğrencinin bilgilerini tutan bir program yazınız. struct Ogrenci adında bir yapı tanımlayınız. Bu yapı içinde:

- no
 - ad
 - ortalama
- alanları bulunsun.,

Programda:

1. 2 elemanlı bir struct Ogrenci dizisi oluşturunuz.
2. Her öğrenci için numara, ad ve ortalama bilgilerini program içinde atayınız.
3. Tüm öğrencileri ekrana yazdırınız.
4. Ortalaması en yüksek olan öğrencinin adını ve ortalamasını yazdırınız.

```
#include <stdio.h>
#include <string.h>
struct Ogrenci {
    int no; char ad[20]; float ortalama;
};

int main(void) {
    struct Ogrenci sinif[2];
    sinif[0].no = 101;
    strcpy(sinif[0].ad, "Ali");
    sinif[0].ortalama = 85.5;
    sinif[1].no = 102;
    strcpy(sinif[1].ad, "Ayse");
    sinif[1].ortalama = 91.0;
    int enlyi = 0;
    printf("Ogrenci Listesi:\n");
    for (int i = 0; i < 2; i++) {
        printf("No: %d | Ad: %s | Ortalama: %.2f\n", sinif[i].no, sinif[i].ad, sinif[i].ortalama);
        if (sinif[i].ortalama > sinif[enlyi].ortalama) {
            enlyi = i;
        }
    }
    printf("\nEn yuksek ortalamali ogrenci:\n");
    printf("Ad: %s | Ortalama: %.2f\n", sinif[enlyi].ad, sinif[enlyi].ortalama); return 0;
}
```

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

1. Yapıyı Fonksiyona Parametre Olarak Göndermek

- Bir öğrencinin tüm bilgilerini ekrana basan bir fonksiyonumuz var.
- Eğer struct kullanılmazsa, fonksiyona 3 farklı değişken (int, char[], float) göndermek gerekirdi.
- struct ile tek bir paket gönderiyoruz.
- Fonksiyona yapının bir kopyası gider. Fonksiyon içinde yapılan değişiklikler orijinal yapıyı bozmaz.

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

1. Yapıyı Fonksiyona Parametre Olarak Göndermek

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char ad[20];
    int yas;
} Personel;
void personelBilgisiGoster(Personel p){
    printf("--- Personel Karti ---\n");
    printf("Ad %s\n", p.ad);
    printf("Yasi: %d\n", p.yas);
}
int main() {
    Personel calisan1;
    strcpy(calisan1.ad, "Can");
    calisan1.yas = 28;
    personelBilgisiGoster(calisan1);
    return 0;
}
```

```
void personelBilgisiGoster(char ad[], int yas) {
    printf("--- Personel Karti ---\n");
    printf("Ad: %s\n", ad);
    printf("Yasi: %d\n", yas);
}
```

```
er(char ad[], char soyad[], int yas,
```

```
önce void personelGoster(Personel p)
```


Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

1. Yapıyı Fonksiyona Parametre Olarak Göndermek

- İçinde sadece tam sayı tipinde no isimli bir üye barındıran Öğrenci adında bir yapı (**struct**) tanımlayınız.
- Parametre olarak bu yapıyı alan ve öğrencinin numarasını ekrana yazdıran yazdir isimli bir fonksiyon oluşturunuz.
- main fonksiyonu içerisinde bu yapıdan bir değişken oluşturup numarasını 101 yapınız ve hazırladığınız fonksiyonu kullanarak ekrana yazdırınız.

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

1. Yapıyı Fonksiyona Parametre Olarak Göndermek

```
#include <stdio.h>
// 1. Yapı Tanımı
struct Ogrenci {
    int no;
};

// 2. Yapıyı parametre olarak alan fonksiyon
void yazdir(struct Ogrenci o) {
    printf("Ogrenci No: %d\n", o.no);
}

int main() {
    // 3. Yapıdan değişken oluşturma ve fonksiyona gönderme
    struct Ogrenci ogr1 = {101};
    yazdir(ogr1);                                //ogr1.no = 101;

    return 0;
}
```

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

2. Struct Döndüren Fonksiyonlar

- C'de fonksiyonlar struct döndürebilir.
- Bu, birden fazla bilgiyi tek seferde geri döndürmek için faydalıdır.
- Bir fonksiyon sadece bir değer değil, koca bir "Öğrenci" paketi döndürebilir.
- Bu, özellikle yeni kayıt oluştururken çok kullanışlıdır.
- Küçük veri yapılarında oldukça pratiktir.
- Büyük yapılarda pointer ile çalışma daha verimli olabilir.

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

2. Struct Döndüren Fonksiyonlar

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char ad[20];
    int yas;
} Personel;
Personel personelOlustur(char isim[], int yasDeğeri) {
    Personel yeniP;
    strcpy(yeniP.ad, isim);
    yeniP.yas = yasDeğeri;
    return yeniP; // Hazırlanan paket main'e
}
int main() {
    Personel calisan1; // Fonksiyondan gelen paketi karşılamak için bir değişken
    calisan1 = personelOlustur("Can", 28); // Fonksiyonu çağırıyoruz ve dönen değeri calisan1'e eşitliyoruz
    printf("Ad: %s, Yasi: %d\n", calisan1.ad, calisan1.yas);
    return 0;
}
```

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

2. Struct Döndüren Fonksiyonlar

- İçinde sadece tam sayı tipinde no isimli bir üye barındıran Oğrenci adında bir yapı (**struct**) tanımlayınız.
- Parametre olarak bir tam sayı (numara) alan ve bu numarayı içine yerleştirip **hazırladığı yapıyı geri döndüren** ogrenciOlustur isimli bir fonksiyon yazınız.
- main fonksiyonu içerisinde bu fonksiyonu 101 değeriyle çağırarak dönen sonucu bir değişkene atayınız ve ekrana yazdırınız.

2. Struct Döndüren Fonksiyonlar

```
#include <stdio.h>
struct Ogrenci {
    int no;
};
// Yapı (struct) döndüren fonksiyon
struct Ogrenci ogrenciOlustur(int gelenNo) {
    struct Ogrenci yeniOgr; // Fonksiyon içinde geçici bir yapı oluşturulur
    yeniOgr.no = gelenNo; // Parametre olarak gelen sayı içine yazılır

    return yeniOgr; // Hazırlanan paket (struct) geri döndürülür
}
int main() {
    struct Ogrenci ogr1; // Fonksiyondan gelen yapıyı karşılamak için bir değişken oluşturma
    ogr1 = ogrenciOlustur(101); // Fonksiyon çağrılır ve dönen "paket" ogr1 değişkenine atanır
    printf("Ogrenci No: %d\n", ogr1.no);
    return 0;
}
```

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

3. Struct Pointer

- Struct türündeki bir değişkenin adresi pointer ile tutulabilir.
- Böylece fonksiyonlar gerçek struct üzerinde işlem yapabilir.
- Orijinal veriyi doğrudan değiştirmek
- Bellek kopyasını önlemek
- Büyük veri yapılarında daha verimli işlem yapmak
- Dinamik bellek ile çalışmak

Örnek kullanım:

- Öğrenci bilgisini güncelleme
- Bağlı liste düğümleri
- Dosya kayıt yapıları

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

3. Struct Pointer

- Struct pointer ile alanlara erişimde iki yöntem vardır:

`(*p).alan`

`p->alan`

- İkinci kullanım daha yaygın ve daha okunaktır.

Örnek:

`(*p).no = 101;`

`p->no = 101;`

- İki ifade aynı işi yapar.

Struct Fonksiyon

Fonksiyon ve Struct Kullanımı

3. Struct Pointer

. ile -> Farkı

- . → normal struct değişkeni için
- -> → struct pointer için

Karşılaştırma:

- ogr1.no = 5; // struct değişkeni
- p->no = 5; // struct pointer

Struct Fonks

Fonksiyon ve Struct Kullanımı

3. Struct Pointer

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char ad[20];
    int yas;
} Personel;
// 1. Parametre olarak struct POINTER (*) alan fonksiyon
void yasiGuncelle(Personel *p, int yeniYas) {
    // Ok (->) operatörü: "p'nin işaret ettiği yerdeki yas değerini değiştir"
    p->yas = yeniYas;
}
int main() {
    Personel calisan1 = {"Can", 28};
    printf("Guncelleme oncesi yas: %d\n", calisan1.yas);
    // 2. Fonksiyona değişkenin adresini (&) gönderiyoruz
    yasiGuncelle(&calisan1, 30);
    printf("Guncelleme sonrasi yas: %d\n", calisan1.yas);
    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char ad[20];
    int yas;
} Personel;
Personel personelOlustur(char isim[], int yasDeğeri) {
    Personel yeniP;
    strcpy(yeniP.ad, isim);
    yeniP.yas = yasDeğeri;
    return yeniP; // Hazırlanan paket main'e
}
int main() {
    Personel calisan1; // Fonksiyondan gelen paketi karşıla
    calisan1 = personelOlustur("Can", 28); // Fonksiyonu çağır
    printf("Ad: %s, Yasi: %d\n", calisan1.ad, calisan1.yas);
}
```

```
#include <stdio.h>
```

```
typedef struct {  
    int yas;  
} Personel;
```

```
// Fonksiyon 'Personel p' şeklinde bir kopya alır
```

```
void yasiGuncelleValue(Personel p) {  
    p.yas = 30; // Sadece kopya değişti!  
}
```

```
int main() {  
    Personel calisan1 = {28};  
    yasiGuncelleValue(calisan1); // Verinin kopyasını gönderdik  
    printf("Value Sonucu: %d\n", calisan1.yas); // ÇIKTI: 28 (Değişmedi)  
    return 0;  
}
```

```
#include <stdio.h>
```

```
typedef struct {  
    int yas;  
} Personel;
```

```
// Fonksiyon 'Personel *p' şeklinde bir ADRES alır
```

```
void yasiGuncelleReference(Personel *p) {  
    p->yas = 30; // Doğrudan orijinal adresteki veri değişti!  
}
```

```
int main() {  
    Personel calisan1 = {28};  
    yasiGuncelleReference(&calisan1); // Adresini (&) gönderdik  
    printf("Reference Sonucu: %d\n", calisan1.yas); // ÇIKTI: 30 (Değişti!)  
    return 0;  
}
```

Struct Fonksiyon

- İçinde sadece tam sayı tipinde no isimli bir üye barındıran Öğrenci adında bir yapı (**struct**) tanımlayınız.
- Parametre olarak **bu yapının adresini (pointer)** ve yeni bir numara (int) alan noGuncelle isimli bir fonksiyon yazınız. Bu fonksiyon, **ok (->)** operatörünü kullanarak öğrencinin numarasını güncellesin.
- main fonksiyonu içerisinde bir öğrenci oluşturup numarasını 101 yapınız. Ardından hazırladığınız fonksiyonu kullanarak bu numarayı 102 olarak güncelleyiniz ve değişikliğin kalıcı olduğunu ekrana yazdırarak kontrol ediniz.

Struct Fonksiyon

```
#include <stdio.h>
struct Ogrenci {
    int no;
};
// Struct Pointer (*) alan fonksiyon
void noGuncelle(struct Ogrenci *o, int yeniNo) {
    o->no = yeniNo; // Ok operatörü (->) ile adresteki veriye erişip güncelliyoruz
}
int main() {
    struct Ogrenci ogr1 = {101}; // Bir yapı değişkeni oluşturuyoruz
    printf("Guncelleme oncesi: %d\n", ogr1.no);
    noGuncelle(&ogr1, 102); // Fonksiyona ogr1'in adresini (&) gönderiyoruz
    printf("Guncelleme sonrasi: %d\n", ogr1.no); // Pointer kullandığımız için ana programdaki veri de değişti
    return 0;
}
```

Struct Fonksiyon

Bir sınıftaki 3 öğrencinin bilgilerini tutan bir program yazınız. Öğrenci adında bir struct tanımlayınız. Bu struct içinde:

- no
 - ad
 - ortalama
- alanları bulunsun.

Programda:

1. 3 elemanlı bir Öğrenci dizisi oluşturunuz.
2. Öğrenci bilgilerini program içinde atayınız.
3. yazdir adında bir fonksiyon yazınız. Bu fonksiyon bir Öğrenci parametresi alsın ve öğrenci bilgilerini ekrana yazdırsın.
4. Döngü ile tüm öğrencileri yazdırınız.

```
#include <stdio.h>
#include <string.h>
struct Ogrenci {
    int no;char ad[20];float ortalama;
};
void yazdir(struct Ogrenci o) {
    printf("No: %d | Ad: %s | Ortalama: %.2f\n", o.no, o.ad, o.ortalama);
}
int main(void) {
    struct Ogrenci sinif[3];
    sinif[0].no = 101;
    strcpy(sinif[0].ad, "Ali");
    sinif[0].ortalama = 85.5;
    sinif[1].no = 102;
    strcpy(sinif[1].ad, "Ayse");
    sinif[1].ortalama = 91.0;
    sinif[2].no = 103;
    strcpy(sinif[2].ad, "Mehmet");
    sinif[2].ortalama = 78.3;
    for (int i = 0; i < 3; i++) {
        yazdir(sinif[i]);
    }
    return 0;
}
```

Struct Fonksiyon

Bir kitabı temsil eden Kitap adlı struct tanımlayınız. Bu struct içinde:

- ad
 - sayfa
 - fiyat
- alanları bulunsun.

Programda:

- 1.kitapOlustur adında bir fonksiyon yazınız. Bu fonksiyon bir Kitap döndürsün.
- 2.indirimYap adında bir fonksiyon yazınız. Bu fonksiyon Kitap * parametresi alsın ve fiyatı 10 azaltıp güncellesin.
- 3.main içinde bir kitap oluşturup önce eski, sonra yeni hâlini yazdırınız.


```
#include <stdio.h>
#include <string.h>
struct Kitap {
    char ad[30]; int sayfa; float fiyat;
};
struct Kitap kitapOlustur(char ad[], int sayfa, float fiyat) {
    struct Kitap k;
    strcpy(k.ad, ad);
    k.sayfa = sayfa;
    k.fiyat = fiyat;
    return k;
}
void indirimYap(struct Kitap *k) {
    k->fiyat -= 10;
}
int main(void) {
    struct Kitap k1 = kitapOlustur("C Programlama", 320, 150);
    printf("Ilk durum: %s | %d sayfa | %.2f TL\n", k1.ad, k1.sayfa, k1.fiyat);
    indirimYap(&k1);
    printf("Indirimli: %s | %d sayfa | %.2f TL\n", k1.ad, k1.sayfa, k1.fiyat);
    return 0;
}
```

Struct

İç İçe Struct Yapıları

- Bir struct içinde başka bir struct alan olarak bulunabilir.
- Böylece daha karmaşık nesneler modellenenebilir.
- Bir kutunun içine, başka bir kutu yerleştirmek gibidir.

```
struct Tarih {  
    int gun, ay, yil;  
};
```

```
struct Ogrenci {  
    int no;  
    char ad[20];  
    struct Tarih dogum;  
};
```

```
struct Ogrenci ogr1;
```

```
ogr1.dogum.gun = 15;  
ogr1.dogum.ay = 4;  
ogr1.dogum.yil = 2005;
```

Struct

Bir öğrencinin doğum tarihini de tutmak istiyoruz.
Önce Tarih adlı bir struct tanımlayınız:

- gun
- ay
- yıl

Daha sonra Öğrenci adlı bir struct tanımlayınız:

- no
- ad
- dogum → Tarih türünde olsun

Programda bir öğrenci oluşturup bilgilerini ekrana yazdırınız.

```
#include <stdio.h>
#include <string.h>
struct Tarih {
    int gun; int ay; int yil;
};
struct Ogrenci {
    int no;
    char ad[20];
    struct Tarih dogum;
};
int main(void) {
    struct Ogrenci ogr;
    ogr.no = 201;
    strcpy(ogr.ad, "Zeynep");
    ogr.dogum.gun = 15;
    ogr.dogum.ay = 4;
    ogr.dogum.yil = 2005;
    printf("No: %d\n", ogr.no);
    printf("Ad: %s\n", ogr.ad);
    printf("Dogum Tarihi: %02d/%02d/%d\n",
        ogr.dogum.gun, ogr.dogum.ay, ogr.dogum.yil);
    return 0;
}
```